Async I/O / Event-Driven I/O

Earlier approaches improved server scalability, but many were still **synchronous**:

- Thread-per-connection
- Non-blocking polling
- `select()` / basic multiplexing

A better model is **asynchronous, event-driven I/O**, where the system tells you **exactly which sockets are ready**, so CPU time is spent doing useful work.

1. Core Idea

Instead of:

```
None
Check every socket repeatedly
```

Use:

```
None
OS notifies ready sockets/events
```

Then the application processes only those ready connections.

2. Why This Is Better

With polling:

```
None
CPU wastes cycles checking sockets with no work
```

With event-driven async I/O:

```
None
CPU wakes only when useful work exists
```

This dramatically improves efficiency.

3. epoll() (Linux)

On Linux, a major API for scalable event-driven networking is:

```
None
epoll()
```

It allows applications to register many file descriptors and receive events when they are ready.

Examples of readiness:

- New connection available
- Socket readable
- Socket writable
- Error / disconnect

4. How epoll Works

Step 1: Register Sockets

Tell kernel which sockets to watch.

```
None
listen socket

client sockets
```

Step 2: Wait for Events

Application calls:

```
None
epoll_wait()
```

Kernel sleeps efficiently until events occur.

Step 3: Ready List Returned

Kernel returns exact ready descriptors:

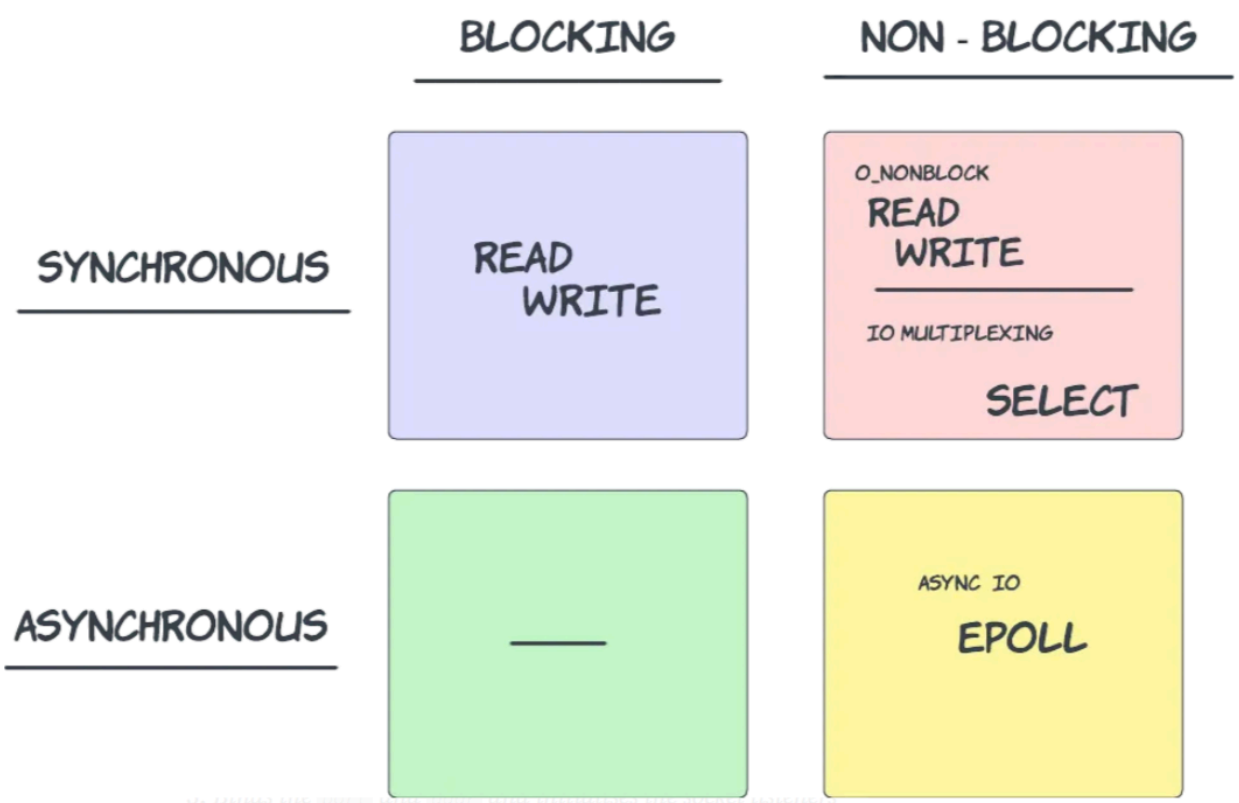
```
None
client_12 readable

client_37 writable

listen_socket new connection
```

Step 4: Process Events

Main thread loops through returned events and handles them.



5. Why It Feels Like Event-Driven Architecture

The application reacts to events:

```
None
connection event

read event

write event

timeout event
```

This is why many modern servers are event-loop based.

6. Why It Scales So Well

✔ No Thread Per Connection

```
None
1 thread can manage thousands of sockets
```

✔ No Busy Waiting

Only wakes when work exists.

✔ Lower Memory Usage

Fewer threads = less stack memory.

✔ High Throughput + Low Latency

Used in high-performance systems.

7. Real-World Software Using Event Loops

- Redis
- NGINX
- Node.js
- HAProxy

8. Synchronous vs Asynchronous

Model	What Happens
Blocking Sync	Wait for I/O
Non-blocking Polling	Keep checking manually

select/poll	Wait for readiness, scan results
epoll Event Loop	Get exact ready events efficiently

9. Important Clarification

Strict OS-level “async I/O” can mean completion-based APIs, but in many system design discussions:

```
None
epoll/event-loop models are commonly grouped under async scalable I/O
```

10. Mental Model

```
None
Don’t ask every socket if it has work.

Let OS tell you which sockets have work.
```

One-Line Summary

Async/event-driven I/O uses mechanisms like epoll to return exact ready sockets, allowing one thread to efficiently process thousands of connections without busy waiting or thread-per-connection overhead.